

Professional Engineering Review

Supreme Fusion Prediction Market Platform

Full-Stack Code, Database, Security & Architecture Audit

Date: May 23, 2026

Reviewer: Z.ai Engineering Team

CRITICAL	HIGH	MEDIUM	LOW	INFO	Total
25	47	49	39	19	179

Total Issues Identified Across All System Layers

This document presents a comprehensive professional engineering review of the Supreme Fusion prediction market platform. The review covers the full technology stack: Next.js frontend and API routes, Supabase database schema and migrations, SQL stored procedures, authentication and authorization, Stripe payment integration, AI swarm trading system, and deployment configuration. Each issue is categorized by severity with specific file references and actionable fixes.

Table of Contents

1. Executive Summary & Systemic Risk Assessment
2. CRITICAL Issues That Must Be Fixed Before Any Deployment
3. Core Infrastructure: Config, Auth & Library Layer
4. API Routes: Authentication Gaps & Financial Integrity
5. Frontend: Security, Data Flow & User Experience
6. Database Layer: Schema, RLS & SQL Functions
7. Payment & Financial System Integrity
8. AI Swarm System
9. Deployment & Configuration
10. Priority Remediation Roadmap

1. Executive Summary & Systemic Risk Assessment

The Supreme Fusion platform is a prediction market application built with Next.js 16, Supabase (PostgreSQL), Stripe, and a custom AI trading swarm powered by Groq. The codebase spans approximately 80+ files across frontend pages, API routes, shared libraries, database migrations, and SQL stored procedures. This review identified **179 issues** across all severity levels, with **25 CRITICAL** issues that represent fundamental threats to system security, data integrity, and financial correctness. In its current state, the platform **must not be deployed to production** without addressing the CRITICAL and HIGH severity items first.

1.1 Systemic Risk Summary

Beyond individual bugs, the codebase exhibits several **systemic architectural issues** that compound the risk of any single vulnerability. These cross-cutting problems are not isolated to one file or module but permeate the entire system, making them especially dangerous and difficult to patch incrementally.

#	Severity	File	Issue	Fix
1	CRITICAL	Cross-Layer	verifyAdminAuth function does not exist. 14 of 20 admin API routes import it and will crash at module load time. This leaves the entire admin panel non-functional.	Add the export to lib/adminAuth.ts with proper JWT/cookie validation, or refactor all routes to use the existing requireAdmin() function.
2	CRITICAL	Cross-Layer	Module-level Supabase client singleton (lib/supabase.ts) used for both client and server code. Server-side operations run under anonymous RLS with no auth context. Admin auth, Stripe webhooks, and AI swarm all import this singleton.	Split into supabase-client.ts (browser) and supabase-server.ts (per-request with cookie context). All server code must use the server client.
3	CRITICAL	Cross-Layer	TOCTOU race conditions in all balance mutation paths. Both adminAdjustBalance and creditUserAccount perform read-then-write on user balances. Concurrent requests can create or destroy money.	Convert all balance mutations to atomic SQL: UPDATE users SET balance = balance + \$amount WHERE id = \$userId via RPC functions.
4	CRITICAL	Database	Users can escalate their own privileges via RLS. The users_update_own policy has no column restriction, allowing any user to SET is_admin = TRUE or modify their own sc_balance via direct SQL.	Add a BEFORE UPDATE trigger that strips protected columns, or split into selective UPDATE policies.

#	Severity	File	Issue	Fix
5	CRITICAL	Database	Duplicate and conflicting SQL function definitions. Two incompatible versions exist for process_stake, resolve_market, apply_referral_reward, and redeem_kyc. The 003 migration versions reference non-existent columns and will crash at runtime.	Delete the 003 versions. Keep only the standalone versions, fix their issues, and ensure they match the schema.
6	CRITICAL	Cross-Layer	OAuth callback never exchanges the code for a session. The exchangeCodeForSession call is commented out. Users are redirected to the dashboard without ever creating a valid session.	Implement the actual code exchange in the callback route.
7	HIGH	Cross-Layer	No rate limiting on any endpoint, including admin login. Brute-force attacks are possible at network speed with no lockout or delay.	Implement rate limiting middleware (e.g., Upstash Ratelimit or Cloudflare rules).
8	HIGH	Cross-Layer	SC-to-USD redemption rate discrepancy of 10x between two redeem_kyc function versions (0.01 vs 0.10). Depending on which function is active, users get 10x more or less than expected.	Standardize on a single rate read from site_config rather than hardcoding.

Table 1: Systemic Cross-Cutting Issues

2. CRITICAL Issues That Must Be Fixed Before Any Deployment

The following issues are show-stoppers. Each one individually represents a scenario where the system either fails to function, leaks sensitive data, or enables financial fraud. No deployment, even in a staging environment with real user data, should proceed until these are resolved.

2.1 Authentication & Authorization Failures

#	Severity	File	Issue	Fix
1	CRITICAL	middleware.ts	Cookie refresh tokens are silently dropped. The setAll callback in middleware only sets cookies on the request object, never on the response. Users will be logged out when their JWT expires because the refreshed token is never sent back to the browser.	Use <code>NextResponse.next()</code> and set cookies on the response object. Follow the official Supabase middleware pattern for Next.js App Router.
2	CRITICAL	lib/adminAuth.ts	Client-side supabase singleton used for admin auth. The module imports the browser-side supabase client (anon key, no cookie context). On the server, this client has no auth session and operates under anonymous RLS policies. Admin verification queries will fail or return incorrect results.	Create a server-side admin Supabase client using the service role key with proper cookie handling.
3	CRITICAL	app/(auth)/callback/route.ts	OAuth code is never exchanged for a session. The <code>exchangeCodeForSession</code> call is commented out. The user is redirected to the dashboard without ever creating a valid Supabase session. They will appear logged in on the client but have no valid session on the server.	Uncomment and implement the code exchange. Also validate the state parameter for CSRF protection.
4	CRITICAL	app/api/admin/login/route.ts	Admin session tokens are trivially forgeable. The token is Base64-encoded <code>timestamp + Math.random()</code> . <code>Math.random()</code> is not cryptographically secure. An attacker who captures any token can immediately understand the format and forge valid tokens.	Use <code>crypto.randomUUID()</code> or signed JWTs with a server-side secret. Never use <code>Math.random()</code> for security tokens.

#	Severity	File	Issue	Fix
5	CRITICAL	app/admin/login/page.tsx	HttpOnly cookies set from client-side JavaScript. The code attempts document.cookie with HttpOnly flag, but browsers silently ignore HttpOnly when set via JavaScript. Tokens are stored as plain JavaScript-accessible cookies, fully exposed to XSS.	The login API must set HttpOnly cookies via Set-Cookie response headers. Never set sensitive cookies from client-side code.

2.2 Financial Integrity Failures

#	Severity	File	Issue	Fix
1	CRITICAL	lib/adminAuth.ts	TOCTOU race condition in adminAdjustBalance. The function reads currentBalance, computes newBalance in JavaScript, then writes it back. Two concurrent admin adjustments will both read the same balance and the second write overwrites the first, creating or destroying money.	Use atomic SQL: UPDATE users SET sc_balance = sc_balance + \$1 WHERE id = \$2 via an RPC function.
2	CRITICAL	lib/stripe.ts	creditUserAccount has the same TOCTOU race condition. Two concurrent webhook deliveries for the same user will both read the same balance and the second write overwrites the first. Users get double-credited or lose credits.	Use atomic SQL UPDATE for all balance mutations.
3	CRITICAL	lib/stripe.ts	createRedemptionPayout drains the platform bank account. stripe.payouts.create() sends real money from the platform bank account to the user. There is no Stripe Connect setup and no validation that the amount matches a redemption request.	Do not use payouts.create() for user redemptions. Use Stripe Connect transfers or a separate payout service that validates against redemption_requests.

2.3 Database Security Failures

#	Severity	File	Issue	Fix
1	CRITICAL	001_initial.sql	Users can update their own admin flags and balances via RLS. The users_update_own policy allows FOR UPDATE USING (auth.uid() = id) with no column restriction. Any user can SET is_admin = TRUE, sc_balance = 999999, or kyc_status = verified on their own row.	Add a BEFORE UPDATE trigger that strips protected columns (is_admin, is_publisher, sc_balance, gc_balance, kyc_status). Or only allow updates via SECURITY DEFINER RPC functions.

#	Severity	File	Issue	Fix
2	CRITICAL	001_initial.sql	Public can see all user data including balances, KYC documents, and password hashes. The users_select_public policy uses USING (TRUE), exposing every column to any unauthenticated reader.	Create a secure view or policy that only exposes safe columns (id, handle, name, user_tier, created_at).
3	CRITICAL	001_initial.sql	Direct position INSERT bypasses process_stake. The positions_insert_own policy allows any authenticated user to INSERT INTO positions directly, bypassing the process_stake() function that deducts balances and adds to market pools.	Remove positions_insert_own policy entirely. Positions should only be created through process_stake() (SECURITY DEFINER).
4	CRITICAL	001_initial.sql	Direct transaction INSERT allows balance manipulation. The transactions_insert_own policy lets any user insert transactions (e.g., type = PAYOUT, amount = 99999), creating a fake audit trail.	Remove this policy. Transactions should only be created inside SECURITY DEFINER functions.
5	CRITICAL	003_rpc_functions.sql	process_stake references non-existent columns: closes_at, trading_volume, entry_price, current_value, pnl, pool_before. None of these exist in the markets or positions tables. The function will crash at runtime.	Rewrite to match the actual schema: expires_at, total_volume_sc, entry_prob, cost, lmsr_subsidy.
6	CRITICAL	003_rpc_functions.sql	resolve_market has SQL injection via EXECUTE format. While %L escapes values, this pattern is fragile and unnecessary for the conditional logic it implements.	Replace with conditional IF/ELSE using direct SQL instead of EXECUTE format.

3. Core Infrastructure: Config, Auth & Library Layer

3.1 Configuration Issues

#	Severity	File	Issue	Fix
1	CRITICAL	next.config.js + .ts	Two conflicting config files exist. Next.js will load one and ignore the other in undefined order. The .js file lacks images.domains; the .ts file lacks any future config added to .js.	Delete next.config.js. Keep only next.config.ts with all configuration merged.
2	HIGH	.gitignore	*.lock excludes lockfiles. Without a committed package-lock.json, npm install produces non-deterministic builds. Version drift can cause deployment failures.	Remove *.lock line from .gitignore. Commit package-lock.json.
3	MEDIUM	package.json	Next.js version ^16.0.0 may not exist as a stable release. This could pull a canary or unstable build.	Pin to the latest stable release (e.g., ^15.1.0).
4	MEDIUM	.env.example	Financial rates exposed as NEXT_PUBLIC_ prefix. NEXT_PUBLIC_SC_USD_RATE, NEXT_PUBLIC_PROTOCOL_FEE_RATE etc. are sent to the browser. Malicious users can modify these client-side.	Remove NEXT_PUBLIC_ prefix; read these server-side only. Pass to client via API if needed.
5	MEDIUM	vercel.json	Hardcoded Vercel org ID in the config file. If committed to a public repo, the org slug is exposed.	Move org ID to environment variable or Vercel project settings.
6	MEDIUM	Cross-file	Environment variable naming inconsistency. Three different names for the app URL: NEXT_PUBLIC_APP_URL, NEXT_PUBLIC_SITE_URL, NEXT_PUBLIC_URL.	Standardize to a single env var name (e.g., NEXT_PUBLIC_APP_URL) across all files.

3.2 Library Layer Issues

#	Severity	File	Issue	Fix
1	HIGH	lib/supabase.ts	signInWithGoogle/Apple use window.location.origin. These functions call window.location.origin which only exists in the browser. If accidentally called server-side during SSR, the app crashes with ReferenceError.	Add 'use client' directive or move to a dedicated client-only module.
2	HIGH	lib/supabase.ts	SQL injection risk in search filter. query.or uses string interpolation: question.ilike.%"\${filters.search}%" with user input directly interpolated.	Use Supabase textSearch() or parameterized .like() with proper escaping of % and _ wildcards.
3	MEDIUM	lib/supabase.ts	getAdminStats misuses .count(). Uses .select(id).count() then checks data?.length. In Supabase v2, .count() returns the count in metadata, not data.	Use { count: exact } option and read from the count field of the response.
4	MEDIUM	lib/utls.ts	Division by zero in calculatePnl. For side === NO, the formula is (entryPrice - currentPrice) * stake / currentPrice. If currentPrice === 0, this produces Infinity or NaN.	Add a guard: if (currentPrice === 0) return 0 or return -stake.
5	HIGH	types/index.ts	Type names do not match database schema. User.gold_balance / User.sweeps_balance do not match DB columns gc_balance / sc_balance. kyc_status: approved does not match the DB value verified.	Align types with the actual database schema. Generate types with supabase gen types.
6	HIGH	lib/stripe.ts	Wrong env var name. Uses NEXT_PUBLIC_URL which does not exist in .env.example or .env.local. This resolves to undefined, producing broken callback URLs.	Standardize to NEXT_PUBLIC_APP_URL and add it to .env.example.
7	HIGH	lib/stripe.ts	Webhook handler trusts unvalidated metadata. session.metadata.userId is used to credit accounts without verifying it matches the authenticated user who initiated the checkout.	Validate that the userId matches the authenticated user. Use Stripe client_reference_id as a secondary check.

4. API Routes: Authentication Gaps & Financial Integrity

The API layer contains the most concentrated cluster of vulnerabilities in the entire system. Fourteen admin routes import a function (verifyAdminAuth) that does not exist, causing them to crash at module load time. Three admin endpoints have zero authentication, exposing sensitive data publicly. Two endpoints allow any unauthenticated user to view any other user financial positions and transactions through IDOR vulnerabilities. The admin login mechanism uses forgeable tokens with no rate limiting, making it trivially exploitable.

4.1 Unauthenticated Endpoints

#	Severity	File	Issue	Fix
1	CRITICAL	api/admin/activity/route.ts	Returns all admin audit logs to any unauthenticated user. No verifyAdminAuth import or check at all. Exposes admin IDs, actions, and change details publicly.	Add verifyAdminAuth check at the top of the handler.
2	CRITICAL	api/admin/stats/route.ts	Returns all platform stats (total users, active users, fees, AI config, recent transactions) to any unauthenticated caller.	Add authentication and authorization.
3	CRITICAL	api/admin/system-status/route.ts	Exposes system internals (DB latency, Stripe/Groq API status, AI swarm config, full site_config) to any unauthenticated caller.	Add authentication and authorization.
4	CRITICAL	api/admin/users/[id]/positions/route.ts	No authentication at all. Any unauthenticated user can query any other user positions by iterating UUIDs. Exposes market positions, stakes, and PnL.	Add verifyAdminAuth and verify the caller is an admin.
5	CRITICAL	api/admin/users/[id]/transactions/route.ts	No authentication at all. Any unauthenticated user can view any user financial transactions by ID. Exposes amounts, currencies, and metadata.	Add authentication.

4.2 Admin Login Security

#	Severity	File	Issue	Fix
1	CRITICAL	api/admin/login/route.ts	No rate limiting on admin login. An attacker can brute-force the admin password at network speed with no attempt counting, lockout, or delay.	Implement rate limiting (e.g., 5 attempts per 15 minutes, exponential backoff).

#	Severity	File	Issue	Fix
2	HIGH	api/admin/login/route.ts	process.env.ADMIN_PASSWORD_HASH = hash does not persist. Writing to process.env at runtime does not persist across serverless function invocations. Every cold start will hit the plaintext fallback path.	Store the hash in the database or a secrets manager, never in process.env at runtime.
3	HIGH	api/admin/login/route.ts	generateLink misused. supabaseAdmin.auth.admin.generateLink({ type: magiclink }) generates a link, not a session. The response shape does not contain { data: { session } }. This code path always falls through to the insecure fallback.	Use admin.generateLink correctly or switch to a proper token generation method.
4	HIGH	api/admin/check/route.ts	Weak admin check with multiple bypass vectors. (a) user_metadata.is_admin is client-writable via updateUser(). (b) Email comparison without verified-email check. (c) Only app_metadata.admin is secure, but it is OR-ed with insecure checks.	Remove the user_metadata check entirely. Require app_metadata.admin === true as the sole authority.

4.3 Financial API Issues

#	Severity	File	Issue	Fix
1	CRITICAL	api/admin/users/[id]/adjust-balance/route.ts	TOCTOU race condition on balance update. Reads current balance, computes new balance in JS, then writes it back. Two concurrent requests create/destroy money.	Use atomic SQL: UPDATE users SET sc_balance = sc_balance + \$amount WHERE id = \$userId via RPC.
2	HIGH	api/admin/markets/[id]/resolve/route.ts	No pre-resolution status check. A market can be resolved regardless of its current status (already resolved, cancelled, etc.). Double-resolution is possible.	Check market.status before calling resolve_market RPC. Reject if not active.
3	HIGH	api/admin/markets/[id]/suspend/route.ts	Non-atomic suspend + refund. The status update and refund_market_positions RPC are separate operations. If the refund RPC fails, the market is marked cancelled but positions are not refunded.	Wrap in a transaction or perform refund first, then update status.
4	HIGH	api/admin/users/route.ts	Sort column injection. The sort parameter from query params is passed directly to .order(sort as any). An attacker can sort by any column including password_hash or email.	Whitelist allowed sort columns.

#	Severity	File	Issue	Fix
5	HIGH	api/fraud-report/route.ts	Import { auth } from @/lib/auth does not exist. lib/auth.ts exports individual named functions, not an auth object. auth.getSession() will throw Cannot read properties of undefined.	Change to import { getSession } from @/lib/auth and call await getSession().

5. Frontend: Security, Data Flow & User Experience

The frontend layer reveals a pattern of simulated functionality that has not been connected to real backend services. Sign-in and sign-up use `setTimeout` to simulate authentication. The KYC form collects no data. Fee rates and financial calculations are performed client-side and can be manipulated. The admin panel relies on client-side auth checks that can be bypassed by disabling JavaScript. Several interactive components are broken or non-functional, including dropdown menus that are always visible and leaderboard category tabs that have no effect.

5.1 Security-Critical Frontend Issues

#	Severity	File	Issue	Fix
1	CRITICAL	admin/login/page.tsx	HttpOnly cookies set from client-side JavaScript. Browsers ignore the HttpOnly flag when set via <code>document.cookie</code> . Admin tokens are fully exposed to any XSS payload.	The login API must set HttpOnly cookies via Set-Cookie response headers.
2	CRITICAL	admin/layout.tsx	Admin panel protected only by client-side check. The only protection is a fetch to <code>/api/admin/check</code> followed by <code>setIsAdmin</code> . If JS is disabled or React state is modified, admin content renders. The admin UI code is shipped to unauthenticated browsers.	Use middleware or server-side session validation to redirect unauthorized users before admin page code is ever sent.
3	CRITICAL	admin/settings/page.tsx	<code>STRIPE_SECRET_KEY</code> referenced in a use client component. Even checking <code>.includes(sk_live)</code> on the client is a security anti-pattern that reveals the key prefix pattern.	Move all env-var inspection to a server API route. Expose only a boolean <code>isStripeLive</code> flag.
4	HIGH	Header.tsx	DropdownMenu always visible with no open/close state. The local DropdownMenu/DropdownMenuContent components have no state management. The dropdown is permanently visible. The imported Radix-style DropdownMenu is shadowed.	Remove local definitions and use the proper DropdownMenu component from <code>@/components/ui/DropdownMenu</code> .
5	HIGH	Dashboard layout	No authentication guards on any dashboard routes. Any unauthenticated user can navigate directly to <code>/dashboard</code> , <code>/portfolio</code> , <code>/markets</code> , etc. The layout uses a hardcoded <code>mockUser</code> object.	Implement auth middleware or HOC that checks session before rendering. Redirect to <code>/signin</code> if unauthenticated.

#	Severity	File	Issue	Fix
6	HIGH	TradeForm.tsx	Financial calculations performed client-side. Fee rates (0.02, 0.01) are hardcoded. PnL is calculated client-side. All of these can be trivially manipulated by the user.	All financial calculations must be performed and validated server-side. The client should only display values returned by the API.

5.2 Functional & UX Issues

#	Severity	File	Issue	Fix
1	HIGH	signin + signup pages	Simulated authentication with no real backend. Both use setTimeout to simulate auth. OAuth buttons also just simulate with setTimeout. Users are redirected to /dashboard without any real credential verification.	Integrate with actual Supabase auth before production.
2	HIGH	kyc/page.tsx	KYC form submits no data. The identity form uses raw input elements with no state variables. handleIdentitySubmit does not collect or send any form data. Document upload and selfie steps have no file input handling.	Add controlled form state, file input handling, and actual API submission.
3	HIGH	PositionCard.tsx	cn() utility used but never imported. Line 44 calls cn() but there is no import statement. This will cause a runtime ReferenceError.	Add import { cn } from @/lib/utls at the top of the file.
4	MEDIUM	Multiple files	Inconsistent age restriction. Footer says 21+ while disclaimer and auth layout say 18+ only. This is a legal compliance issue for a gambling/prediction market platform.	Standardize to the legally required age across all locations.
5	MEDIUM	history/page.tsx	dateRange filter not applied. The dateRange state populates a select but is never referenced in the filteredTransactions useMemo. Selecting Last 7 Days, etc. has no effect.	Add date filtering logic to the filteredTransactions computation.
6	MEDIUM	leaderboard/page.tsx	Category tabs are non-functional. selectedCategory state exists but filteredLeaderboard useMemo does not filter by category. All three tabs show the same data.	Implement category filtering in the useMemo computation.

#	Severity	File	Issue	Fix
7	MEDIUM	layout.tsx	Toaster provider not in app tree. The Toaster component and useToast hook are implemented but never added to the component tree in layout.tsx. Calls to useToast() will throw must be used within a ToastProvider.	Wrap {children} with Toaster in the root layout.
8	MEDIUM	Multiple pages	No loading states on multiple pages. markets/[id], portfolio, and leaderboard render immediately with mock data. When real API calls are integrated, there will be a flash of empty content.	Add loading.tsx files or inline loading states with Suspense boundaries.

6. Database Layer: Schema, RLS & SQL Functions

The database layer contains the most structurally dangerous issues in the entire system. Row-Level Security policies are misconfigured to allow privilege escalation and direct data manipulation. Multiple stored procedures reference columns that do not exist in the actual schema, meaning they will crash at runtime. Two incompatible versions of every core financial function exist, with different parameters, different fee calculations, and in one case a 10x discrepancy in the monetary conversion rate. Financial columns lack CHECK constraints that would prevent negative balances and zero-amount stakes.

6.1 Schema Constraint Failures

#	Severity	File	Issue	Fix
1	HIGH	001_initial.sql	No CHECK constraint: balances can go negative. gc_balance and sc_balance have DEFAULT 0 and NOT NULL but no CHECK (gc_balance >= 0). A race condition or bug could create negative balances.	Add CHECK (gc_balance >= 0) and CHECK (sc_balance >= 0).
2	HIGH	001_initial.sql	No CHECK constraint: stake can be zero or negative. positions.stake NUMERIC(16,2) NOT NULL has no minimum check.	Add CHECK (stake > 0).
3	HIGH	001_initial.sql	No constraint: probabilities do not sum to 1. yes_probability and no_probability are independent NUMERIC(5,4) fields with no CHECK that they sum to 1.0.	Add CHECK (yes_probability + no_probability = 1) or compute one from the other.
4	HIGH	001_initial.sql	total_traders can be inflated. total_traders INTEGER increments on every stake, not per unique user. A user who stakes 10 times adds 10 to the count.	Only increment if no existing position for this user+market. Use a count of distinct users from positions.
5	MEDIUM	001_initial.sql	resolution CHECK includes NULL inside IN. CHECK (resolution IN (yes, no, invalid, cancel, NULL)) does not work as intended. In SQL, value IN (... , NULL) does not match NULL.	Use: CHECK (resolution IS NULL OR resolution IN (yes, no, invalid, cancel)).
6	MEDIUM	001_initial.sql	Missing RLS on 10+ tables. No RLS enabled on: categories, publishers, publisher_earnings, achievements, social_shares, market_comments, signals, leaderboard_cache, ai_agent_trades, gc_packages, amoe_entries.	Enable RLS and add appropriate policies for all tables, especially financial and PII tables.

6.2 SQL Function Integrity Issues

#	Severity	File	Issue	Fix
1	CRITICAL	process_stake.sql	Not SECURITY DEFINER. The function runs as the calling user, but RLS policies on positions and transactions may block operations. It updates users and markets tables which may also be blocked.	Add SECURITY DEFINER and ensure the function is the only way to create positions/transactions.
2	HIGH	process_stake.sql	LMSR uses REAL type causing floating point precision loss. v_exp REAL on line 114 causes precision loss in the LMSR calculation. Financial calculations must use exact NUMERIC arithmetic.	Remove v_exp REAL declaration. Use NUMERIC throughout. EXP() and LN() on NUMERIC are available in PostgreSQL.
3	HIGH	resolve_market.sql	Any authenticated user can resolve markets. GRANT EXECUTE ON FUNCTION resolve_market TO authenticated combined with no admin verification inside the function. Any user can resolve any market and pay themselves.	Add admin check inside the function and change GRANT to admin-only role.
4	HIGH	redeem_position.sql	Market pool can go negative. The market pool is reduced by v_redemption_value without checking if the pool has sufficient funds. Concurrent redemptions can drive the pool below zero.	Add check: IF (pool - v_redemption_value) < 0 THEN RAISE EXCEPTION. Lock the market row with FOR UPDATE.
5	HIGH	redeem_position.sql	Probability not recalculated after redemption. After reducing the pool, the function does not call update_market_probability(). Stale probabilities mislead other traders.	Add probability recalculation after pool adjustment.
6	HIGH	redeem_position.sql	No lock on market row. FOR UPDATE OF p only locks the position row. The market row is not locked, so concurrent stakers can change pools during redemption calculation.	Add FOR UPDATE OF m to also lock the market row.
7	HIGH	apply_referral_reward.sql	Referred user not locked. Only the referrer is locked with FOR UPDATE. The referred user referral_bonus_claimed flag is updated without a lock, creating a TOCTOU race.	Lock the referred user row with FOR UPDATE before updating.
8	HIGH	create_user_with_referral.sql	Insecure referral code generation. RANDOM() is not cryptographically secure. The code is predictable if you know the handle (first 6 chars from the handle).	Use gen_random_uuid() for the code.
9	MEDIUM	002_ai_swarm.sql	Invalid SQL comments. Lines use // (JavaScript-style) instead of -- (SQL). These will cause parse errors.	Replace // with --.

7. Payment & Financial System Integrity

The payment system has the most severe consequences for real-world impact. The Stripe integration contains a function that can drain the platform bank account directly, webhook handlers that trust unvalidated metadata, and race conditions that can create or destroy virtual currency. The financial calculations in the database use REAL (floating point) types instead of NUMERIC, which introduces rounding errors that accumulate over millions of transactions. The SC-to-USD redemption rate differs by 10x between two versions of the same function, which could result in users receiving 10x more or less real money than expected.

#	Severity	File	Issue	Fix
1	CRITICAL	lib/stripe.ts	createRedemptionPayout drains platform balance. stripe.payouts.create() creates a payout from the connected account balance (the platform by default). This sends real money from the platform bank account with no Stripe Connect setup.	Do not use payouts.create() for user redemptions. Use Stripe Connect transfers or a separate payout service.
2	CRITICAL	lib/stripe.ts	TOCTOU race condition in creditUserAccount. Read-then-write on user balances. Concurrent webhooks double-credit users or lose credits.	Use atomic SQL UPDATE via RPC function.
3	HIGH	lib/stripe.ts	Import at bottom of file. import { supabase } from ./supabase is at line 208. This is the client-side singleton. Webhook handlers (server-side) should never use it.	Move import to top. Replace with a server-side Supabase client using the service role key.
4	HIGH	lib/stripe.ts	Transaction inserts not atomic. Promise.all([gcTxPromise, scTxPromise]). If one insert fails, the other may succeed, leaving an inconsistent transaction log. The balance was already updated.	Wrap in a Supabase RPC function that does balance update + transaction inserts in a single transaction.
5	HIGH	webhooks/stripe/route.ts	No validation of gc_amount from metadata. parseInt(session.metadata.gc_amount) could be NaN, 0, or negative. A manipulated checkout session metadata could cause a negative coin credit.	Validate: gcAmount > 0 and isFinite(gcAmount).
6	HIGH	webhooks/stripe/route.ts	KYC auto-verify from webhook trusts session.metadata.user_id. If Stripe session metadata is manipulable, an attacker could get KYC verified for any user.	Verify the user_id against the original verification request stored in DB.
7	CRITICAL	redeem_kyc.sql vs 003 version	SC-to-USD rate discrepancy of 10x. The standalone version uses 0.01 while 003 uses 0.10. Depending on which function is active, users either get 10x more or 10x less than expected in real money.	Standardize on a single rate read from site_config rather than hardcoding.

#	Severity	File	Issue	Fix
8	MEDIUM	lib/stripe.ts	Outdated Stripe API version (2023-10-16). May be missing features or have breaking changes.	Update to the latest stable API version and test.
9	MEDIUM	lib/stripe.ts	Hardcoded package prices in cents. If the frontend displays different prices, users will be charged the wrong amount.	Source prices from a shared config or database.

8. AI Swarm System

The AI swarm is an automated trading system that uses Groq LLM to generate trading decisions. The system has several issues ranging from LLM prompt injection to CORS misconfiguration and missing authentication on the tick endpoint. The AI agents can be manipulated through malicious market questions injected into the LLM prompt, and the tick endpoint is accessible to anyone who can reach the URL, allowing unauthorized trading or denial-of-service attacks.

#	Severity	File	Issue	Fix
1	HIGH	ai-swarm/groq.ts	LLM prompt injection via market question. The market question field is interpolated directly into the Groq prompt. A malicious question like Ignore previous instructions. Return [...] could manipulate AI trading decisions.	Sanitize market data before inclusion in prompt. Use structured input (JSON) rather than template string. Add explicit prompt boundaries.
2	CRITICAL	ai-swarm-tick/index.ts	CORS allows all origins. Access-Control-Allow-Origin: * means any website can trigger the AI swarm tick endpoint, potentially causing unwanted trades or DoS.	Restrict to your domain or use Supabase internal auth.
3	CRITICAL	ai-swarm-tick/index.ts	Stack trace exposed in production. error.stack is returned in the HTTP response, leaking internal implementation details.	Remove error.stack from production responses. Log it server-side only.
4	HIGH	ai-swarm-tick/index.ts	No authentication on the endpoint. Anyone who can reach the Edge Function URL can trigger it. The function only checks an env var for enabled/disabled, not caller identity.	Verify Authorization header against service role key or use Supabase auth.
5	HIGH	ai-swarm-tick/index.ts	execution_price stores confidence, not price. execution_price: decision.confidence is semantically wrong. The execution price should be the actual cost/probability at which the trade was executed.	Store result?.cost or the market probability at execution time.
6	HIGH	ai-swarm-cron.sql	Service role key placeholder in SQL. The cron job contains Authorization: Bearer your-service-role-key. If replaced with the actual key, it would be stored in plaintext visible to anyone with pg_catalog access.	Use current_setting(app.service_role_key) or a secrets manager.
7	MEDIUM	lib/ai-swarm.ts	getAgentStats loads all trades into memory. Selects ALL rows from ai_agent_trades with no limit, then aggregates in JavaScript. As the table grows, this will OOM the server.	Use a Supabase RPC or database view that performs aggregation in SQL.

#	Severity	File	Issue	Fix
8	MEDIUM	lib/ai-swarm.ts	Hardcoded AI user ID. 00000000-0000-0000-0000-000000000001 is hardcoded. If the database row changes, this silently queries the wrong user.	Move to an environment variable AI_USER_ID.

9. Deployment & Configuration

#	Severity	File	Issue	Fix
1	CRITICAL	supabase/config.toml	Default JWT secret in production. <code>jwt_secret = your-jwt-secret-change-in-production</code> . If deployed without changing, all JWTs can be forged, granting full database access.	Generate a cryptographically random secret and set it via environment variable.
2	HIGH	supabase/config.toml	Placeholder OAuth credentials. Google and Apple client IDs/secrets are placeholder values that would be exposed if committed with real values.	Use environment variable substitution for OAuth credentials.
3	MEDIUM	supabase/config.toml	<code>double_confirm_changes = false</code> . Email changes do not require double confirmation, allowing account takeover if an attacker gains session access.	Set to true.
4	HIGH	.gitignore	<code>*.lock</code> excludes lockfiles. Without committed <code>package-lock.json</code> , npm install produces non-deterministic builds across environments.	Remove <code>*.lock</code> line. Commit <code>package-lock.json</code> .
5	MEDIUM	package.json	Missing <code>@types/bcrypt</code> in <code>devDependencies</code> . TypeScript compilation will fail or require <code>skipLibCheck: true</code> to mask the error.	Add <code>@types/bcrypt</code> to <code>devDependencies</code> .
6	MEDIUM	.env.example	<code>ADMIN_PASSWORD</code> in plaintext. <code>OVERVIEW.txt</code> describes using <code>ADMIN_PASSWORD_HASH</code> (bcrypt), yet the example shows a plaintext password. Encourages storing plaintext.	Replace <code>ADMIN_PASSWORD</code> with <code>ADMIN_PASSWORD_HASH</code> and add generation instructions.
7	MEDIUM	Multiple files	Three separate config systems: <code>site_config</code> , <code>ai_swarm_config</code> , and <code>ai_swarm_settings</code> all store overlapping configuration. This causes confusion about which is authoritative.	Consolidate into a single <code>site_config</code> table and remove <code>ai_swarm_config</code> and <code>ai_swarm_settings</code> .
8	MEDIUM	ai-user-seed.sql	<code>ON CONFLICT</code> resets SC balance to 100000. Every time this migration runs, it resets the AI user balance, potentially overwriting legitimate gains/losses.	Use <code>ON CONFLICT (id) DO NOTHING</code> or only set balance if below a threshold.

10. Priority Remediation Roadmap

Based on the severity and interdependence of the issues found, the following remediation roadmap organizes fixes into four phases. Each phase should be completed and validated before proceeding to the next. The estimated effort assumes a single senior full-stack engineer working full-time.

Phase 1: Critical Security & Authentication (Week 1-2)

1. Fix middleware cookie handling. Follow the official Supabase middleware pattern to write refreshed cookies to the NextResponse object. This is the root cause of users being logged out.
2. Add verifyAdminAuth to lib/adminAuth.ts. This single function unblocks 14 admin routes. Implement JWT/cookie validation with app_metadata.admin === true as the sole authority check.
3. Fix OAuth callback. Uncomment exchangeCodeForSession and implement proper state parameter validation for CSRF protection.
4. Fix admin login security. Replace forgeable Base64 tokens with signed JWTs. Add rate limiting. Remove process.env write hack. Set HttpOnly cookies from the server.
5. Fix RLS policies. Remove users_update_own column-unrestricted UPDATE. Remove positions_insert_own and transactions_insert_own. Add BEFORE UPDATE trigger to strip protected columns.
6. Add authentication to the 5 unauthenticated endpoints: activity, stats, system-status, users/[id]/positions, users/[id]/transactions.
7. Generate and set a real JWT secret in Supabase config. Remove placeholder values.

Phase 2: Financial Integrity (Week 2-3)

1. Convert all balance mutations to atomic SQL. Every path that modifies user balances must use UPDATE users SET balance = balance + \$amount via RPC functions. This eliminates TOCTOU race conditions in adminAdjustBalance, creditUserAccount, and the balance adjustment API.
2. Remove stripe.payouts.create() from user redemptions. Implement Stripe Connect or a separate payout service that validates against redemption_requests before sending money.
3. Consolidate duplicate SQL functions. Delete the 003 migration versions of process_stake, resolve_market, apply_referral_reward, and redeem_kyc. Fix and keep only the standalone versions.
4. Fix column references in SQL functions. Audit every function against the actual schema and fix mismatches (closes_at to expires_at, trading_volume to total_volume_sc, etc.).
5. Add SECURITY DEFINER to all financial functions. process_stake, redeem_position, resolve_market, and redeem_kyc must run with elevated privileges to bypass RLS during legitimate operations.
6. Add CHECK constraints for non-negative balances, positive stakes, and probability sums.
7. Standardize SC-to-USD rate. Read from site_config instead of hardcoding. Remove the 10x discrepancy.

8. Validate Stripe webhook metadata. Cross-reference userId with authenticated user and client_reference_id.

Phase 3: Architecture & Data Flow (Week 3-4)

1. Split lib/supabase.ts into client and server modules. This is the root cause of issues where server code uses the browser singleton with no auth context.
2. Delete next.config.js. Keep only next.config.ts with all configuration merged.
3. Remove *.lock from .gitignore. Commit package-lock.json for deterministic builds.
4. Standardize environment variable naming. Use NEXT_PUBLIC_APP_URL consistently across all files.
5. Move financial calculations server-side. Remove hardcoded fee rates from client components. Fetch from API.
6. Integrate real authentication. Replace all setTimeout simulations in signin/signup with actual Supabase auth calls.
7. Fix admin panel server-side auth. Use middleware or server-side session validation instead of client-side checks.
8. Remove NEXT_PUBLIC_ prefix from sensitive financial rate env vars.

Phase 4: Quality & Polish (Week 4-5)

1. Add rate limiting middleware across all API endpoints, prioritizing admin login and fraud reporting.
2. Implement loading states and Suspense boundaries across all dashboard pages.
3. Fix DropdownMenu component. Remove local shadow definitions, use proper open/close state.
4. Add Toaster provider to root layout.
5. Implement actual KYC form data collection and submission.
6. Add debounce to all search inputs.
7. Fix leaderboard category filtering and history date range filtering.
8. Add missing indexes for common query patterns (positions.is_settled, transactions.created_at, etc.).
9. Standardize age restriction across all locations.
10. Generate Supabase types with supabase gen types and align TypeScript types with actual schema.
11. Add missing RLS policies on all tables that currently lack them.
12. Implement proper error handling in all SQL functions and API routes.

This review was conducted as a static analysis of the provided source code. No dynamic testing, penetration testing, or runtime verification was performed. The findings represent potential vulnerabilities and defects based on code inspection. A follow-up security audit with dynamic testing is strongly recommended after the Phase 1 and Phase 2 fixes are complete.